Pique — Socios, No Socios y Precio por Jugador

> Relevamiento del estado actual, decisiones, diseño y plan de implementación para que un club gestione su padrón de socios, que los socios ingresen a la app con su DNI y que cada jugador pague según su condición (socio / no socio) y la modalidad de juego (ej. tenis single / dobles).

Fecha de referencia: 2026-10-13
Estado: Propuesta / Validación parcial con el club (ver §13)
Alcance: Backend (Prisma, auth, precios, reservas, cuentas), Admin, App del jugador

---

Índice

- 0. Resumen ejecutivo
- 1. Requerimientos del club
- 2. Relevamiento: qué tenemos hoy
- 3. Decisiones de diseño
- 4. Experiencia del jugador (app)
- 5. Experiencia del admin
- 6. Diseño técnico
- 7. Importación del padrón
- 8. Reglas de negocio
- 9. Casos borde
- 10. Plan de implementación
- 11. Riesgos
- 12. Criterios de aceptación
- 13. Preguntas al club

0. Resumen ejecutivo

Hoy el sistema **no tiene el concepto de socio**, el precio de un turno es **por cancha** (un único número), y el jugador **solo puede ingresar con email**. El club necesita:

1. **Padrón de socios** administrado por el club (alta, baja, suspensión, categoría).
2. **Cuenta en la app para cada socio**, creada por nosotros a partir del padrón, con **login por DNI**.
3. **Precio por jugador**: cada jugador paga según sea socio o no. Ej.: no socio $6.000, socio $3.000.
4. **Modalidades con precio propio**: en tenis, single (2 jugadores) y dobles (4 jugadores). Los precios los administra el club.
5. Todo lo anterior **funcionando igual desde la app del jugador y desde el admin**.
6. **Cobro en el mostrador con posnet o QR integrado**: que el pago de cada jugador quede registrado solo en el sistema, sin que el cajero lo cargue a mano.

Decisiones principales:

- **Socio = registro de padrón (`ClubMember`) sobre la ficha del cliente.** La condición de socio nunca la declara quien reserva: sale siempre del padrón.
- **Cuenta del socio pre-creada y vinculada desde el inicio.** Al importar el padrón se crean `User` + `Client` + `ClubMember` ya conectados. El socio **activa** su cuenta con su DNI y un código que le llega por WhatsApp.
- **Login por DNI o email**, con DNI único entre usuarios.
- **Precio por jugador como modo de precios del club** (`pricingMode = PER_PLAYER`), opcional y apagado por defecto. No reemplaza el precio por cancha que usan los demás clubes; no se generaliza más allá de lo que este club necesita.
- **Modalidades de juego** (`ActivityFormat`: Single 2, Dobles 4, Pádel 4…) con su tabla de precio por jugador por categoría.
- **El precio socio de un acompañante solo se aplica cuando ese socio confirma desde su propia cuenta** (o lo valida el staff en el mostrador). Así nadie puede usar el DNI de un socio para pagar menos.
- **DNI opcional para no socios.** Los socios ingresan con DNI porque viene del padrón; los no socios se registran como hoy y pueden sumar su DNI si quieren ingresar con él. La seguridad del precio socio no depende de que los no socios den su DNI.
- **Sin pago online para este club.** Se cobra en el mostrador; cada jugador paga su parte.
- **Cobro integrado con la terminal del club**: Caja envía el monto de la parte del jugador a la terminal (posnet o QR), y cuando el pago se aprueba se registra solo, imputado al ítem de ese jugador. Diseño independiente del proveedor; la primera integración depende de qué terminal tenga el club (§6.7).

---

1. Requerimientos del club

#	Requerimiento	Fuente
R1	Administrar socios y no socios	Pedido inicial
R2	Precios y beneficios distintos para socios	Pedido inicial
R3	Cada jugador paga según su condición, no por cancha	P4, P11
R4	Tenis tiene single y dobles con precios distintos; los precios los administra el club	P12

#	Requerimiento	Fuente
R5	Los socios tienen cuenta en la app, creada a partir del padrón (lista manual que ya tenemos)	Reunión
R6	Login por DNI	Pedido del club
R7	Debe funcionar en la app del jugador, no solo en el admin	Reunión
R8	La cuota social no se cobra con Pique por ahora (posible a futuro)	P3
R9	Pago solo en el mostrador (posnet o QR). Sin seña; en el mejor caso cada jugador paga su parte	P16
R10	Integrar el sistema con las terminales de cobro del club para que los pagos se registren automáticamente en la reserva	P16
R11	Recargo por luces encendidas (ya existe)	P2
R12	No socios se registran igual que hoy; pedir DNI se percibe como invasivo	P17

---

2. Relevamiento: qué tenemos hoy

2.1. Identidad y acceso del jugador

Pieza	Dónde	Estado	Impacto
`User.email`	`schema.prisma` `model User`	**Obligatorio y único.** Se usa en ~34 lugares del backend (13 archivos).	Un socio sin email no puede tener cuenta hoy.
`User.password`	idem	Obligatorio	Una cuenta pre-creada necesita un estado "pendiente de activar".
`User.phoneNumber`	idem	Obligatorio	Sirve como canal para el código de activación.
`User.dni`	idem	**Opcional, sin índice único, sin normalizar**	No se puede usar para login tal como está.
Login	`AuthController.ts` `login`	**Solo email + contraseña.** También magic link por email y OAuth (Google/Apple/Facebook).	Hay que aceptar DNI.
Registro	`AuthController.register`	Email obligatorio, DNI opcional	Hay que exigir DNI (al menos para este club) y evitar duplicados.
Recuperar contraseña	—	**No existe**	Imprescindible si el socio entra con DNI + contraseña.

Pieza	Dónde	Estado	Impacto
Reclamo de perfil de club	`UserClubProfileService.ts` `claimClubProfile`	Vincula `User` ↔ `Client` si coincide email, teléfono o **DNI**	⚠️ El DNI del `User` lo carga el propio usuario sin verificar. Con socios, alguien podría registrarse con el DNI de un socio, reclamar su ficha y pagar precio socio. Hay que cerrarlo (§6.4).
WhatsApp	`WhatsappTemplateCategory.AUTHENTICATION`, `wpp-service`	Existe infraestructura y la categoría de plantilla de autenticación	Canal natural para el código de activación.

2.2. Personas en el club

Pieza	Estado	Impacto
`Client` (ficha por club)	Estable: `name`, `dni`, `phone`, `email`, `isProfessor`, `userId?`	Base del socio.
`Membership`	Rol de acceso (OWNER/ADMIN/STAFF/CUSTOMER)	**No es "socio".** No reusar el nombre.
`ClientRelationship`	Existe (Academia)	Futuro grupo familiar.
Importación masiva	**No existe**	Hay que construirla para el padrón.

2.3. Precios y reservas

Pieza	Estado	Impacto
`PricingService.calculateCourtPrice` + `CourtPriceRule`	Precio **por cancha** según día y franja	Se mantiene para los clubes en modo por cancha.
Cálculo duplicado	Base + duración + luces está copiado en 4 lugares de `BookingService.ts` (`quoteBookingPrice` L2554, `createBooking` L3399, `getAvailableSlotsWithCourts` L7286, resumen financiero L3161)	Hay que centralizar antes de agregar otro modo de precio.
`ActivityType`	Duraciones y horarios, **sin cantidad de jugadores ni modalidades**	Single / dobles no existe.
`BookingParticipant`	Existe. En admin siempre se resuelve a un `Client`. En la app se invita por usuario y el invitado acepta. Se agregan **después** de crear la reserva.	Base para el precio por jugador.

Pieza	Estado	Impacto
`BookingBillingConfig` (`SHARED`)	Reparte un total fijo entre participantes; la suma debe coincidir con `Booking.price`	Con precio por jugador el total cambia según quién juegue; hay que adaptarlo.
Cuenta de la reserva	**Un** `AccountItem` BOOKING por reserva	Necesitamos un ítem por jugador.
Modal de pagos	Presets `FULL` , `COURT_ONLY` , `CUSTOM_ITEMS`	`CUSTOM_ITEMS` ya permite cobrar ítems puntuales: sirve para cobrarle a cada jugador lo suyo.
Descuentos	`DiscountPolicy` por cliente; **no se aplican online ni cuando reserva STAFF** (`applyDiscount=false`)	El precio socio no puede ser un descuento. Se modela como precio.

2.4. Cobros

Pieza	Estado	Impacto
`Payment`	Tiene `method` (CASH, TRANSFER, CARD, OTHER), `channel` (incluye `CARD_TERMINAL` y `VIRTUAL_WALLET`), `source` (POS, ONLINE, BACKOFFICE), `externalReference` , `payerParticipantRef` / `coveredParticipantRef` y `idempotencyKey` .	**Ya soporta** registrar quién pagó y por quién, y una referencia externa de la terminal.
`PaymentAllocation`	Imputa un pago a ítems puntuales de la cuenta	Sirve para imputar el pago de un jugador a su ítem.
`ClubPaymentIntegration`	Conexión OAuth del club con MercadoPago (tokens cifrados, estado)	Reutilizable para cobro presencial con MercadoPago. `PaymentProvider` solo tiene `MERCADO_PAGO` .
`OnlinePaymentAttempt` + webhook	`OnlineCheckoutService.ts` : valida firma, bloquea la fila, verifica monto, es idempotente y crea el `Payment`	**Buen patrón para copiar**, pero está atado a `Booking` + checkout online + `userId` del jugador.
Integración con terminales físicas	**No existe**	Hay que construirla (§6.7).

2.5. Brechas

#	Brecha	Impacto
G1	No existe socio, categoría, estado ni historial	Bloqueante
G2	Un usuario no puede existir sin email	Bloqueante (socios sin email)
G3	No hay login por DNI y el DNI no es único	Bloqueante (R6)
G4	No hay activación de cuentas pre-creadas ni recuperación de contraseña	Bloqueante

#	Brecha	Impacto
G5	El reclamo de perfil por DNI permite suplantar a un socio	Bloqueante de seguridad
G6	El precio es por cancha; no hay precio por jugador	Bloqueante (R3)
G7	No existen modalidades (single / dobles)	Bloqueante (R4)
G8	La cuenta tiene un solo ítem de cancha	Bloqueante (cobro por jugador)
G9	Cálculo de precio duplicado en 4 lugares	Riesgo técnico alto
G10	No hay importación masiva	Alto (puesta en marcha)
G11	No hay forma de habilitar funcionalidades por club desde el backend (`CLUB_LOCKED_SIDEBAR_ITEMS` es solo frontend y hardcodeado)	Medio
G12	No hay integración con terminales de cobro presencial	Alto (R10)

---

3. Decisiones de diseño

D1. Socio como entidad propia (`ClubMember`), 1:1 con `Client`

No como flag en `Client`: necesitamos estado, categoría, número de socio, fechas e historial. "No socio" = el cliente no tiene `ClubMember` o no está activo y vigente.

D2. La condición de socio sale siempre del padrón

Ni el jugador en la app ni el staff en el mostrador "marcan" a alguien como socio al reservar. La categoría de cada jugador se calcula desde `ClubMember`. Si alguien dice ser socio y no figura, se lo da de alta en el padrón y se recalcula.

D3. Socios con cuenta pre-creada y vinculada

Al importar el padrón se crea, para cada socio, en una sola operación:

```
User (estado PENDING_ACTIVATION, sin contraseña utilizable)
  └ Client del club (userId = ese User)
      └ ClubMember (categoría, número, estado)
```

El vínculo `User ↔ Client` queda hecho desde el origen. **No depende de que el socio "reclame" su perfil**, que es la parte insegura del flujo actual.

D4. Login por DNI

- El campo de login acepta **DNI o email**. Si son solo dígitos → DNI; si tiene `@` → email.
- `User.dni` se normaliza (solo dígitos) y pasa a ser **único** entre usuarios.
- `User.email` pasa a ser **opcional** (sigue siendo único cuando existe). Es el cambio con más superficie (~34 usos): todo lo que asume email (magic link, notificaciones, OAuth, MercadoPago payer) tiene que tolerar su ausencia.

- Se aplica a toda la plataforma (no es un login especial del club): es inofensivo para los demás y evita tener dos sistemas de login.

Alternativa descartada: generar emails falsos (`dni@socios.pique`). Rompe magic link, notificaciones y recuperación, y ensucia datos globales.

D4b. DNI de los no socios: opcional (P17)

El DNI no es necesario para la seguridad del precio socio. La condición de socio sale del padrón y de la cuenta pre-creada (D3). No sale del DNI que escribe un usuario. Por eso no hace falta exigírselo a los no socios.

Quién	DNI	Cómo ingresa
Socio	Viene del padrón (obligatorio ahí)	DNI + contraseña
No socio	**Opcional** al registrarse o después, desde su perfil	Email + contraseña, Google/Apple/Facebook, o DNI si lo cargó

Para que no se sienta invasivo:

- En el registro, el campo dice "DNI (opcional) — para ingresar con tu DNI y que el club te identifique más rápido".
- El DNI completo **nunca se muestra** a otros jugadores. En búsquedas solo se muestra el nombre abreviado; en el admin se muestra completo solo al personal del club.
- La política de privacidad (`privacy.tsx`) ya menciona el DNI entre los datos que se recopilan. Igual conviene que la revise quien corresponda para confirmar que la finalidad (identificación y acceso) esté explícita.

Si un no socio carga un DNI que ya tiene una cuenta (por ejemplo, el de un socio), se aplica RN-11: no se le asigna. El ofrecimiento de activar o recuperar va al teléfono del titular de esa cuenta.

Riesgo aceptado: un no socio registrado sin DNI que después se hace socio puede terminar con dos cuentas (la suya y la pre-creada del padrón). La importación lo detecta por teléfono o email y lo marca como **conflicto** para unirlo a mano. Ver E2.

D5. Activación de la cuenta del socio por código

La contraseña no la elegimos nosotros ni el club.

```
1. Socio entra a la app → "Primer ingreso / Activar cuenta" → ingresa su DNI
2. Si el DNI es de una cuenta PENDING_ACTIVATION → se envía un código de 6 dígitos
   por WhatsApp al teléfono del padrón (se muestra enmascarado: "+54 9 11 ****-**78")
3. Ingresa el código → elige su contraseña → cuenta ACTIVE → sesión iniciada
```

- Si el teléfono del padrón no es correcto: el admin genera un código desde la ficha del socio y se lo da en el mostrador.
- El mismo mecanismo sirve para **"Olvidé mi contraseña"** por DNI (hoy no existe).
- Alternativa descartada: contraseña inicial = DNI o número de socio. El DNI no es secreto: cualquiera que conozca el DNI de un socio podría entrar antes que él.

D6. Modo de precios por club: `PER_COURT` (actual) o `PER_PLAYER`

El precio por jugador es como trabaja **este** club, no un requerimiento general. Por eso:

- `ClusterSettings.pricingMode` con default `PER_COURT`. Todos los clubes actuales siguen exactamente igual.
- En `PER_PLAYER`, el precio de la reserva es la suma de lo que paga cada jugador según su categoría y la modalidad.
- No construimos combinaciones generales (precio por cancha por categoría, prorrateos, etc.). Si otro club lo necesita, se evalúa en ese momento.

D7. Modalidades de juego (`ActivityFormat`)

Una actividad tiene una o más modalidades, cada una con su cantidad de jugadores:

Actividad	Modalidad	Jugadores
Tenis	Single	2
Tenis	Dobles	4
Pádel	Pádel	4

Se modela como hijo de `ActivityType` (no como actividades separadas) porque single y dobles comparten canchas, horarios y disponibilidad.

D8. Tabla de precios por jugador

El club administra un precio por jugador para cada combinación **modalidad × categoría** (con "No socio" como categoría base). El precio no varía por día ni horario; solo se suma el recargo por luces (D10b):

Modalidad	No socio	Socio Activo	Socio Cadete
Tenis Single	$6.000	$3.000	$2.500
Tenis Dobles	$5.000	$2.500	$2.000
Pádel	$6.000	$3.000	$3.000

Ejemplo, tenis dobles con 1 socio activo y 3 no socios: $2.500 + 3 × $5.000 = **$17.500**.

D9. Precio socio de un acompañante: requiere confirmación

Riesgo: tres no socios reservan cargando el DNI de socios conocidos y pagan la mitad. Regla:

- **El titular** que reserva desde su cuenta → su categoría se aplica directo (está autenticado).
- **Un acompañante socio** → su precio socio se aplica cuando **confirma desde su propia cuenta** (acepta la invitación). Hasta entonces, su lugar figura a precio no socio con la leyenda "Precio socio pendiente de confirmación".
- **En el mostrador**, el staff puede validar a un socio presencialmente (con carnet o DNI) y se registra quién lo validó.

D10. Congelamiento del precio

- La parte de cada jugador se calcula cuando se suma (o se confirma) y queda guardada en el participante.
- El total de la reserva cambia mientras se agregan, quitan o confirman jugadores.
- La parte de un jugador que ya tiene pagos imputados **no se modifica nunca**.
- Reservas completadas o canceladas no se recalculan.

D10b. Recargo por luces en precio por jugador (P2)

El precio por jugador **no cambia por día ni horario**. Lo único que varía es el recargo por luces, que ya existe (`ClubSettings.lightsEnabled/lightsExtraAmount/lightsFromHour`) y hoy se suma al precio de la cancha.

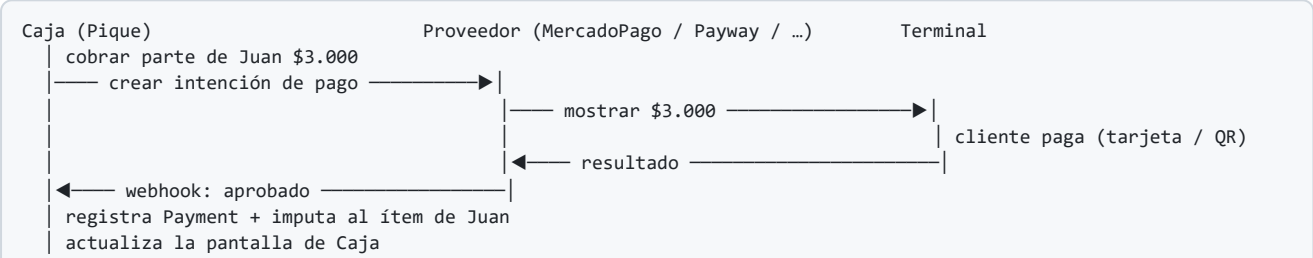
En `PER_PLAYER` , el recargo se calcula igual que hoy: monto fijo por turno desde la hora configurada. **Se reparte en partes iguales entre los lugares** y se suma a la parte de cada jugador, sin importar si es socio. *(Confirmar con el club, P2b: puede que el recargo sea un monto por jugador y no por turno.)*

Ejemplo, dobles con luces ($4.000 de recargo por turno): cada lugar suma $1.000.

Consecuencia: `PlayerPriceRule` **no necesita** día ni franja horaria. Se simplifica a modalidad × categoría → precio.

D12. Cobro integrado con la terminal del club (P16)

Hay un principio clave: **el cobro siempre arranca en Pique, no en la terminal.**



Si el cajero teclea el monto directamente en el posnet, Pique **no tiene forma confiable** de saber a qué reserva y a qué jugador corresponde ese pago. Adivinar por monto y hora es frágil: hay muchos turnos del mismo precio a la misma hora. Por eso:

- El cobro integrado se inicia desde la Caja de Pique, eligiendo los ítems a cobrar (la parte de un jugador, varias partes, o la cuenta completa).
- El monto viaja a la terminal: el cajero no lo tipea, y eso elimina errores.
- Cuando el proveedor confirma el pago, se crea el `Payment` con `method` CARD o TRANSFER (QR), `channel` CARD_TERMINAL o VIRTUAL_WALLET, `source` POS, la referencia del proveedor, el turno de caja abierto y la imputación al ítem del jugador.
- Si el cajero cobró igual "por fuera" (terminal sin conexión, otra terminal), se sigue registrando a mano como hoy.

Viabilidad según la terminal: **no todas las terminales se pueden integrar.** Un posnet tradicional (equipo independiente, sin API) no puede recibir montos desde otro sistema. Se necesita una terminal o un servicio del proveedor que exponga una API de cobro presencial y notifique el resultado. Antes de comprometer el alcance hay que relevar qué terminal y qué proveedor usa el club (P19):

Situación del club	Qué se puede hacer
Terminal o QR de **MercadoPago**	Es el camino más corto: ya tenemos OAuth del club con MercadoPago, cliente HTTP y webhook firmado. Hay que verificar en la documentación vigente de MercadoPago la API de cobro presencial para la terminal y el QR que tenga el club.
Terminal de otro proveedor (Payway, Getnet, Fiserv/Clover, etc.)	Depende de que ese proveedor ofrezca API o SDK de integración para su terminal y de que el club tenga habilitado ese servicio. Hay que consultar al proveedor.

Situación del club	Qué se puede hacer
Posnet sin integración posible	Cobro semiautomático: el cajero registra el pago en Pique con un clic, ya precargado con el monto de la parte y con el número de cupón o de operación. A futuro se puede conciliar con el archivo de liquidación del proveedor.

El diseño interno es **independiente del proveedor** (§6.7): sumar un proveedor nuevo es escribir un adaptador, sin tocar Caja ni reservas.

D11. Nomenclatura

Negocio	Técnico
Socio	`ClubMember`
Categoría de socio	`MemberCategory`
Modalidad (single, dobles)	`ActivityFormat`
Precio por jugador	`PlayerPriceRule`
Modo de precios del club	`ClubSettings.pricingMode`

En la UI: "Socio", "No socio", "Categoría", "Modalidad".

---

4. Experiencia del jugador (app)

4.1. Ingreso

```
Pantalla de ingreso
  [ DNI o email        ]
  [ Contraseña         ]
  ( Ingresar )
  ¿Primera vez? Activá tu cuenta  ·  Olvidé mi contraseña
```

Caso	Qué pasa
Socio, primera vez	"Activá tu cuenta" → DNI → código por WhatsApp → elige contraseña (D5).
Socio que intenta "Ingresar" sin haber activado	Mensaje: "Tu cuenta de socio todavía no está activada" + botón para activar.
No socio nuevo	"Crear cuenta" igual que hoy (nombre, email, teléfono, contraseña, o Google/Apple/Facebook). DNI **opcional** (D4b).
No socio que se registra con un DNI que ya existe	No se crea otra cuenta: "Ese DNI ya tiene una cuenta. Si sos vos, activala o recuperá tu contraseña" → código al teléfono registrado. **Esto impide que alguien tome el DNI de un socio.**
Usuarios existentes que entran con email	Siguen igual.

4.2. Reservar

```
1. Elige actividad y modalidad          Tenis → [Single] [Dobles]
2. Elige día, hora y cancha             Grilla con "Tu precio: $3.000 (socio)"
3. Arma el partido (lugares según la modalidad)
     Lugar 1  Vos — Socio Activo                        $2.500
     Lugar 2  [+ Agregar jugador]   Invitado            $5.000
     Lugar 3  [+ Agregar jugador]   Invitado            $5.000
     Lugar 4  [+ Agregar jugador]   Invitado            $5.000
                                    Total estimado     $17.500
4. Confirma / paga según la política del club
```

Agregar jugador (lugares 2..N):

- **Buscar** a alguien con cuenta (por DNI, email o teléfono, el dato completo y exacto; no se puede buscar por nombre parcial para evitar enumerar usuarios): se le envía la invitación. Mientras no acepte, su lugar se cobra como no socio. Cuando acepta, si es socio, su parte baja y el total se actualiza (D9).
- **Invitado sin cuenta** (solo nombre): no socio.
- **Dejar vacío**: no socio. Se puede completar después.

Privacidad: al buscar por DNI, la app muestra solo el nombre abreviado ("Juan P.") y **no** dice si es socio. La categoría de cada persona la ve solo esa persona y el admin. A los demás jugadores se les muestra el monto de cada lugar, pero no la categoría. *(Validar con el club, P13.)*

4.3. Lo que ve cada jugador después

Quién	Ve
Titular	Todos los lugares, el monto de cada uno, el total y lo pendiente de confirmar.
Invitado	Su invitación con "Tu parte: $2.500 (precio socio)" antes de aceptar.
Socio con la membresía suspendida o vencida	Precio no socio, con la leyenda "Tu membresía no está vigente, consultá en el club".

4.4. Pago

Este club no cobra online ni pide seña (P16). La app reserva y muestra cuánto paga cada uno; el cobro se hace en el mostrador. Por lo tanto:

- El checkout de MercadoPago queda **apagado** para el club. La reserva se confirma según `bookingConfirmationMode` sin pago previo.
- En la app, cada jugador ve "Tu parte: $3.000 — se abona en el club" y, después de jugar, si su parte figura pagada o pendiente.
- El pago online por jugador queda fuera de alcance. Si el club lo pide más adelante, se apoya en los mismos ítems por jugador.

---

5. Experiencia del admin

Pantalla	Cambio
Ajustes > Socios (nuevo)	ABM de categorías. Activar el módulo.
Ajustes > Actividades	Modalidades por actividad (nombre, cantidad de jugadores).
Ajustes > Precios (modo `PER_PLAYER`)	Tabla modalidad × categoría, como la de D8. El recargo por luces se sigue configurando donde está hoy. Reemplaza la grilla por cancha para este club.
Clientes	Badge "Socio · Activo · N° 1234" / "No socio". Filtros. Sección Socio en la ficha: alta, categoría, suspender, reactivar, baja, historial. Estado de la cuenta de la app (pendiente / activa) y botón **"Generar código de activación"**.
Clientes > Padrón	Listado de socios + **Importar padrón**.
Agenda	Al crear la reserva: elegir modalidad → lista de lugares con nombre, badge de categoría y monto; total en vivo. Botón "Validar socio en mostrador" para lugares con precio socio pendiente (D9). Lugares sin asignar visibles en el bloque del turno.
Caja	Un ítem de cancha por jugador. Cobrar a cada uno su parte con "Personalizado" (`CUSTOM_ITEMS`). Nuevo método **"Cobrar con terminal"**: elegir terminal → el monto se envía → la pantalla espera el resultado (aprobado / rechazado / cancelado / expirado) → el pago queda registrado e imputado solo. Opción de reintentar o cancelar la intención.
Ajustes > Cobros (nuevo)	Conectar el proveedor (MercadoPago ya tiene OAuth), dar de alta las terminales y los QR del club (nombre, identificador del dispositivo), y asociarlas a una caja (`CashRegister`).

Respetar las reglas del proyecto: no componentizar `agenda-playground2.tsx` ni `pagos-playground.tsx` como parte de este trabajo.

---

6. Diseño técnico

6.1. Modelo de datos

```
// ---------- Socios ----------

enum ClubMemberStatus {
  ACTIVE
  SUSPENDED
  INACTIVE
}

model MemberCategory {
  id          String   @id @default(cuid())
  clubId      Int
  club        Club     @relation(fields: [clubId], references: [id], onDelete: Cascade)
  code        String
  name        String
  description String?
  sortOrder   Int      @default(0)
  isActive    Boolean  @default(true)
  createdAt   DateTime @default(now()) @db.Timestamptz(3)
  updatedAt   DateTime @updatedAt @db.Timestamptz(3)

  members          ClubMember[]
  playerPriceRules PlayerPriceRule[]

  @@unique([clubId, code])
}

model ClubMember {
  id           String           @id @default(cuid())
  clubId       Int
  club         Club             @relation(fields: [clubId], references: [id], onDelete: Cascade)
  clientId     String           @unique
  client       Client           @relation(fields: [clientId], references: [id], onDelete: Restrict)
  categoryId   String
  category     MemberCategory   @relation(fields: [categoryId], references: [id], onDelete: Restrict)
  memberNumber String?
  status       ClubMemberStatus @default(ACTIVE)
  joinedAt     DateTime         @db.Date
  validUntil   DateTime?        @db.Date   // "al día hasta"; carga manual mientras la cuota no esté en Pique
  leftAt       DateTime?        @db.Date
  statusReason String?
  notes        String?
  createdByUserId Int?
  updatedByUserId Int?
  createdAt    DateTime @default(now()) @db.Timestamptz(3)
  updatedAt    DateTime @updatedAt @db.Timestamptz(3)

  history ClubMemberHistory[]

  @@unique([clubId, memberNumber])
  @@index([clubId, status])
}

model ClubMemberHistory {
  id              String            @id @default(cuid())
  clubMemberId    String
  clubMember      ClubMember        @relation(fields: [clubMemberId], references: [id], onDelete: Cascade)
  clubId          Int
  action          String            // CREATED | STATUS_CHANGED | CATEGORY_CHANGED | UPDATED | MERGED
  fromStatus      ClubMemberStatus?
  toStatus        ClubMemberStatus?
  fromCategoryId  String?
  toCategoryId    String?
  reason          String?
  changedByUserId Int?
  createdAt       DateTime          @default(now()) @db.Timestamptz(3)
```

```
  @@index([clubMemberId, createdAt])
}

// ---------- Identidad ----------

enum UserAccountStatus {
  ACTIVE
  PENDING_ACTIVATION   // creada por un club, todavía sin contraseña elegida
  DISABLED
}

model User {
  // ...campos actuales
  email         String?            @unique        // pasa a opcional
  dni           String?            @unique        // normalizado: solo dígitos
  accountStatus UserAccountStatus  @default(ACTIVE)
  createdByClubId Int?                            // club que pre-creó la cuenta (auditoría)
}

model UserVerificationCode {
  id          String   @id @default(cuid())
  userId      Int
  purpose     String   // ACTIVATION | PASSWORD_RESET
  codeHash    String
  channel     String   // WHATSAPP | ADMIN_ISSUED | EMAIL
  attempts    Int      @default(0)
  expiresAt   DateTime @db.Timestamptz(3)
  consumedAt  DateTime? @db.Timestamptz(3)
  issuedByUserId Int?  // si lo generó un admin en el mostrador
  createdAt   DateTime @default(now()) @db.Timestamptz(3)

  @@index([userId, purpose, consumedAt])
}

// ---------- Precios por jugador ----------

enum ClubPricingMode {
  PER_COURT
  PER_PLAYER
}

model ClubSettings {
  // ...campos actuales
  membersEnabled Boolean         @default(false)
  pricingMode    ClubPricingMode @default(PER_COURT)
  onlineCheckoutEnabled Boolean  @default(true)   // false para este club (P16)
}

model ActivityFormat {
  id             String       @id @default(cuid())
  clubId         Int
  activityTypeId Int
  activityType   ActivityType @relation(fields: [activityTypeId], references: [id], onDelete: Cascade)
  name           String       // "Single", "Dobles"
  playerCount    Int          // 2, 4
  sortOrder      Int          @default(0)
  isActive       Boolean      @default(true)

  priceRules PlayerPriceRule[]

  @@unique([activityTypeId, name])
}

model PlayerPriceRule {
  id               String          @id @default(cuid())
  clubId           Int
  activityFormatId String
  activityFormat   ActivityFormat  @relation(fields: [activityFormatId], references: [id], onDelete: Cascade)
  memberCategoryId String?         // null = no socio
  memberCategory   MemberCategory? @relation(fields: [memberCategoryId], references: [id], onDelete: Cascade)
  pricePerPlayer   Decimal         @db.Decimal(10, 2)
  createdAt        DateTime        @default(now()) @db.Timestamptz(3)
  updatedAt        DateTime        @updatedAt @db.Timestamptz(3)
```

```
  // Sin franjas horarias (P2): el precio solo varía por luces, que ya existe en ClubSettings.
  // Unicidad modalidad + categoría (incluida la fila "no socio" con memberCategoryId null)
  // se valida en el servicio, porque Postgres no considera iguales dos NULL en un @@unique.
  @@index([activityFormatId, memberCategoryId])
}

// ---------- Reserva y cuenta ----------

model Booking {
  // ...campos actuales
  activityFormatId String?    // modalidad elegida
  playerCount      Int?       // snapshot de la modalidad
  // listPrice = todos los lugares a precio no socio; price = Σ partes
  memberBenefitAmount Decimal @default(0) @db.Decimal(10, 2)
}

enum MemberPriceValidation {
  NOT_MEMBER          // no socio o lugar vacío
  SELF                // es el titular autenticado
  ACCEPTED            // aceptó la invitación desde su cuenta
  STAFF_VALIDATED     // validado en el mostrador
  PENDING             // socio agregado por otro, todavía no confirmó → precio no socio
}

model BookingParticipant {
  // ...campos actuales
  slotIndex               Int?
  pricingMemberCategoryId String?
  memberPriceValidation   MemberPriceValidation?
  memberValidatedByUserId Int?
  shareListAmount         Decimal? @db.Decimal(10, 2)  // precio no socio de ese lugar
  shareAmount             Decimal? @db.Decimal(10, 2)  // precio aplicado
  sharePricedAt           DateTime? @db.Timestamptz(3)

  @@unique([bookingId, slotIndex])
}

model AccountItem {
  // ...campos actuales
  bookingParticipantId String?   // un ítem BOOKING por lugar
  clientId             String?
}

// ---------- Cobro presencial integrado ----------

enum PaymentProvider {
  MERCADO_PAGO
  // + el proveedor de la terminal del club, si no es MercadoPago (P19)
}

model PaymentTerminal {
  id               String          @id @default(cuid())
  clubId           Int
  integrationId    String                      // ClubPaymentIntegration del proveedor
  provider         PaymentProvider
  kind             String                      // CARD_TERMINAL | QR
  name             String                      // "Posnet mostrador", "QR caja 1"
  externalDeviceId String                      // id del dispositivo o del punto de cobro en el proveedor
  cashRegisterId   String?                     // caja a la que pertenece
  isActive         Boolean         @default(true)
  createdAt        DateTime        @default(now()) @db.Timestamptz(3)
  updatedAt        DateTime        @updatedAt @db.Timestamptz(3)

  @@unique([provider, externalDeviceId])
  @@index([clubId, isActive])
}

enum TerminalPaymentIntentStatus {
  CREATED       // creada en Pique
  SENT          // aceptada por el proveedor, esperando al cliente en la terminal
  APPROVED      // pago aprobado y registrado en Pique
  REJECTED
```

```
  CANCELLED     // cancelada por el cajero o en la terminal
  EXPIRED
  ERROR
}

model TerminalPaymentIntent {
  id                 String   @id @default(cuid())
  clubId             Int
  terminalId         String
  accountId          String
  bookingId          Int?
  amount             Decimal  @db.Decimal(10, 2)
  itemAllocations    Json     // [{ accountItemId, amount }] — qué partes cubre
  payerParticipantRef String?
  cashShiftId        String?
  createdByUserId    Int
  status             TerminalPaymentIntentStatus @default(CREATED)
  externalReference  String   @unique           // lo que viaja al proveedor para reconocer el pago
  providerIntentId   String?
  providerPaymentId  String?
  paymentId          String?  @unique           // Payment creado al aprobarse
  failureReason      String?
  rawProviderData    Json?
  createdAt          DateTime @default(now()) @db.Timestamptz(3)
  updatedAt          DateTime @updatedAt @db.Timestamptz(3)

  @@index([terminalId, status])
  @@index([accountId, status])
}
```

6.2. Resolución del precio de un lugar

```
pricePerPlayer(format, category):
  1. PlayerPriceRule de (format, category)
  2. si category ≠ null y no hay regla → se usa la de "no socio" (sin beneficio)
  3. si no hay regla de no socio → error de configuración (no se puede reservar esa modalidad)

lightsPerSlot = (luces aplican al horario del turno ? lightsExtraAmount : 0) / playerCount   // D10b

parte(lugar):
  categoría efectiva = categoría del jugador SOLO si memberPriceValidation ∈ {SELF, ACCEPTED, STAFF_VALIDATED}
                       y su ClubMember está ACTIVE y vigente a la fecha del turno
  shareListAmount    = pricePerPlayer(format, null)              + lightsPerSlot
  shareAmount        = pricePerPlayer(format, categoría efectiva) + lightsPerSlot

Booking.price               = Σ shareAmount  (− descuentos personales, si hubiera)
Booking.listPrice           = Σ shareListAmount
Booking.memberBenefitAmount = Σ (shareListAmount − shareAmount)
```

- Redondeo del reparto de luces: a 2 decimales; la diferencia de centavos va al lugar 1.
- Ajuste por duración: no aplica en `PER_PLAYER` (el precio es por jugador por turno de la modalidad).
- La regla "luces desde las HH:MM" se reutiliza tal cual, desde la función centralizada de Fase 0.

6.3. Centralizar el cálculo (Fase 0)

`PricingService.resolveBookingPrice(...)` como **única** entrada para la grilla de turnos, la cotización, la creación de la reserva y el resumen financiero. Internamente decide según `pricingMode`:

- `PER_COURT` → la lógica actual (base + duración + luces), sin cambios de resultado (tests de regresión).
- `PER_PLAYER` → §6.2.

6.4. Auth

Cambio	Detalle
`POST /auth/login`	Acepta `{ identifier, password }` (y `email` por compatibilidad). DNI → normalizar → buscar por `dni`. Cuentas `PENDING_ACTIVATION` → error `ACCOUNT_PENDING_ACTIVATION` para que la app ofrezca activar.
`POST /auth/activation/request`	`{ dni }` → si existe cuenta pendiente, envía código por WhatsApp. **Responde siempre igual** exista o no el DNI (no revelar quién es socio).
`POST /auth/activation/confirm`	`{ dni, code, password }` → activa y emite sesión.
`POST /auth/password-reset/request` y `/confirm`	Mismo mecanismo para cuentas activas (código por WhatsApp o email).
`POST /auth/register`	Igual que hoy. `dni` opcional; si viene y ya existe → `DNI_ALREADY_REGISTERED` con camino a activar/recuperar (el código va al teléfono del titular).
Perfil del jugador	Poder agregar o cambiar el DNI después, con la misma validación de unicidad.
`claimClubProfile`	Dejar de usar **solo** el DNI declarado como señal suficiente para vincular a un `Client` que tenga `ClubMember`. Para socios, el vínculo viene de la pre-creación (D3) o de un admin.
Rate limiting	Login, activación y reset con límite por IP **y por DNI**. Códigos: 6 dígitos, 10 minutos, máximo 5 intentos.
Plantilla WhatsApp	Plantilla de categoría `AUTHENTICATION` en Meta (requiere aprobación previa; contemplar el tiempo en el plan).
Email opcional	Revisar los ~34 usos de `user.email` (notificaciones, magic link, MercadoPago, OAuth linking) para que toleren `null`.

Migración de datos previa a `@unique` en `dni`: normalizar los DNI existentes y **detectar duplicados** entre usuarios. Los duplicados se resuelven a mano antes de aplicar el índice.

6.5. Reservas y participantes

- `createBooking` (modo `PER_PLAYER`): exige `activityFormatId`. Crea `playerCount` lugares: lugar 1 = titular (`SELF` si es el usuario autenticado; si reserva el admin, `STAFF_VALIDATED`), el resto vacíos (`NOT_MEMBER`).
- Agregar jugador (admin o app): ocupa un lugar libre, resuelve `Client` / `User`, setea `PENDING` si es socio agregado por otro, y recalcula solo ese lugar.
- `acceptBookingInvitation`: si el invitado es socio → `ACCEPTED` → recalcula su lugar.
- `validateMemberAtCounter` (nuevo, STAFF/ADMIN): `PENDING` → `STAFF_VALIDATED`, con auditoría.
- Quitar jugador / `leavePlayerBooking`: el lugar vuelve a vacío salvo que su ítem tenga pagos.
- Cuenta: un `AccountItem` BOOKING por lugar ("Cancha — Juan P. (Socio Activo)"). Se mantiene la creación lazy de la cuenta: los ítems se generan a partir de los lugares cuando se crea la cuenta y se sincronizan después.
- `BookingBillingConfig`: en `PER_PLAYER`, el reparto por defecto sale de los lugares. Revisar la validación "suma = `Booking.price`" porque el total ahora cambia al editar jugadores.
- Revisar todo lo que asume un único ítem BOOKING (`ensureBookingAccountWithChargeTx` usa `findFirst({ type: 'BOOKING' })`, resúmenes, reportes, preset `COURT_ONLY`).
- Turnos fijos: la serie guarda la modalidad; cada ocurrencia nace con el titular y lugares vacíos (o los jugadores fijos de la serie, P14).

6.6. Endpoints nuevos (admin)

Método	Ruta	Rol
GET/POST/PATCH	`/:slug/admin/member-categories`	ADMIN (GET también STAFF)
GET	`/:slug/admin/members`	ADMIN
POST/PATCH	`/:slug/admin/clients/:clientId/member`	ADMIN
POST	`/:slug/admin/clients/:clientId/member/status`	ADMIN
GET	`/:slug/admin/clients/:clientId/member/history`	ADMIN
POST	`/:slug/admin/clients/:clientId/activation-code`	ADMIN, STAFF
POST	`/:slug/admin/members/import` (preview / commit)	ADMIN
GET/POST/PATCH	`/:slug/admin/activities/:id/formats`	ADMIN
GET/PUT	`/:slug/admin/player-prices`	ADMIN
POST	`/bookings/:id/participants/:participantId/validate-member`	ADMIN, STAFF

Búsquedas existentes (`person-search` , `participants-search` , `clients-list`): devolver `member` y buscar por número de socio.

6.7. Cobro presencial integrado

Adaptador por proveedor

```
interface InPersonPaymentProvider {
  createIntent(input: {
    terminal: PaymentTerminal;
    amount: number;
    externalReference: string;   // TerminalPaymentIntent.id
    description: string;         // "Cancha 3 — 19:00 — Juan P."
  }): Promise<{ providerIntentId: string }>;

  cancelIntent(terminal: PaymentTerminal, providerIntentId: string): Promise<void>;

  // Se llama desde el webhook y también desde un polling de respaldo
  fetchResult(providerIntentId: string): Promise<{
    status: 'PENDING' | 'APPROVED' | 'REJECTED' | 'CANCELLED' | 'EXPIRED';
    providerPaymentId?: string;
    approvedAmount?: number;
    method?: 'CARD' | 'TRANSFER';   // tarjeta o QR / billetera
    raw: unknown;
  }>;

  validateWebhook(req: unknown): boolean;
}
```

Primera implementación: la del proveedor del club (P19). Si es MercadoPago, reutiliza `ClubPaymentIntegration` , `MercadoPagoService` y la validación de firma del webhook.

Flujo

1. **Caja → "Cobrar con terminal"**: se eligen los ítems (ej. la parte de Juan) y la terminal. Se crea `TerminalPaymentIntent` (`CREATED`) con el monto, la imputación por ítem y el turno de caja abierto. Requiere turno de caja abierto, igual que los cobros manuales.

2. Se llama a `createIntent` → `SENT`. La pantalla queda esperando ("Esperando pago en Posnet mostrador...") con botón **Cancelar**.
3. **Webhook** del proveedor (y un **polling de respaldo** cada pocos segundos mientras la pantalla está abierta, por si el webhook se demora): - Bloquear la fila de la intención (`FOR UPDATE`), como hace hoy `OnlineCheckoutService` . - Si ya está `APPROVED` → no hacer nada (idempotente). - Si el proveedor dice aprobado y **el monto coincide** → crear `Payment` (`source: POS` , `channel: CARD_TERMINAL` o `VIRTUAL_WALLET` , `externalReference` , `payerParticipantRef` , `cashShiftId`) + `PaymentAllocation` a los ítems + asiento contable, en la misma transacción. Estado `APPROVED` . - Si el monto no coincide → `ERROR` + alerta en Caja para revisión manual (no se registra el pago). - Rechazado, cancelado o vencido → se actualiza el estado; el cajero puede reintentar.
4. La pantalla de Caja se actualiza: la parte de Juan figura pagada.

Reglas

ID	Regla
RN-T1	Una terminal solo puede tener **una** intención `SENT` a la vez. Una nueva cancela la anterior o se bloquea.
RN-T2	El monto de la intención no puede superar el saldo pendiente de los ítems elegidos.
RN-T3	Mientras un ítem tiene una intención `SENT` , no se puede cobrar a mano ni incluir en otra intención (evita doble cobro).
RN-T4	Un pago aprobado en el proveedor que llega **después** de que el cajero canceló la intención en Pique se registra igual (el dinero entró) y se marca para revisión.
RN-T5	Las devoluciones de pagos cobrados por terminal siguen el flujo de `Refund` existente. La devolución en el proveedor se hace desde su panel hasta que se integre (fuera de alcance).
RN-T6	Los tokens del proveedor se guardan cifrados como hoy (`accessTokenEnc`).

Lo que hay que relevar antes de implementar (P19)

- Marca y modelo de la terminal, y proveedor o adquirente (MercadoPago, Payway, Getnet, Fiserv, etc.).
- Si el QR que usan es de MercadoPago, de otra billetera o interoperable, y si es un QR fijo (impreso) o se genera por cobro.
- Si el proveedor ofrece API de cobro presencial y si el club la tiene habilitada (cuenta, credenciales, contrato).
- Cuántas terminales y cajas tienen.

---

7. Importación del padrón

CSV/planilla con:

```
nombre,apellido,dni,telefono,email,numero_socio,categoria,estado,fecha_alta,al_dia_hasta
```

Por cada fila, en una transacción por lote:

1. Normalizar DNI y teléfono.

2. Buscar `User` por DNI y `Client` del club por DNI → teléfono → email.
3. Crear o completar: `User` (`PENDING_ACTIVATION` si no existía) → `Client` vinculado → `ClubMember`.
4. Clasificar el resultado: *nuevo*, *existente actualizado*, *conflicto* (DNI de un usuario ya vinculado a otro cliente, número de socio repetido, etc.), *error* (sin DNI, categoría inexistente, teléfono inválido).

Dos pasos: **preview** (no escribe nada, devuelve el reporte) y **commit**. Para la primera entrega alcanza con un script en `apps/backend/scripts/` que corremos nosotros; la pantalla de importación puede venir después.

El DNI es **obligatorio** en el padrón (es la credencial de login). El teléfono es necesario para la activación por WhatsApp. Si falta, el socio se activa con un código generado en el mostrador.

---

8. Reglas de negocio

ID	Regla
RN-01	Socio vigente = `ClubMember.status = ACTIVE`, categoría activa y (`validUntil` null o ≥ fecha del turno).
RN-02	La categoría de un jugador nunca se elige a mano; sale del padrón (D2).
RN-03	Un socio agregado por otro jugador paga precio no socio hasta que confirma desde su cuenta o lo valida el staff (D9).
RN-04	Precio de la reserva = Σ partes de los lugares. Lugar vacío o invitado sin cuenta = precio no socio.
RN-05	La parte de un lugar con pagos imputados no se modifica.
RN-06	Reservas completadas o canceladas no se recalculan.
RN-07	Cambiar la condición de un socio en el padrón no altera partes ya calculadas; un ADMIN puede recalcular las partes sin pagos.
RN-08	Un jugador no puede ocupar dos lugares de la misma reserva. No se pueden superar los lugares de la modalidad.
RN-09	Alta, baja, suspensión y cambio de categoría: solo ADMIN, con motivo para suspender o dar de baja, y registro en historial y auditoría.
RN-10	Generar un código de activación: ADMIN o STAFF, auditado.
RN-11	Un DNI = una cuenta. El registro con un DNI existente deriva a activación o recuperación.
RN-12	Las respuestas de activación y recuperación no revelan si un DNI existe ni si es socio.
RN-13	Con `pricingMode = PER_COURT` o `membersEnabled = false`, todo funciona como hoy.
RN-14	No se puede borrar un cliente socio ni fusionar dos clientes socios sin resolverlo primero.
RN-15	No se puede desactivar una categoría o una modalidad que tenga socios o precios en uso sin reasignarlos.

9. Casos borde

ID	Caso	Comportamiento
E1	Socio sin teléfono o con teléfono viejo	Activa con código generado en el mostrador.
E2	Socio que ya tenía cuenta en Pique con email (y otro DNI o sin DNI)	El import detecta el `User` por DNI; si no hay DNI, por email/teléfono → **conflicto** para revisar a mano, nunca se fusiona solo.
E3	Dos usuarios existentes con el mismo DNI	Se detectan en la migración previa al índice único; se resuelven a mano.
E4	Un no socio intenta registrarse con el DNI de un socio	Se deriva a activación: el código va al teléfono del socio, no al suyo (RN-11).
E5	Socio agregado como acompañante que nunca confirma	Paga precio no socio salvo que el staff lo valide en el mostrador.
E6	Jugador ya pagó su parte y se baja	Su parte no se toca; el lugar se libera recién cuando se resuelve el pago (devolución o traspaso).
E7	Socio identificado después de haber pagado como no socio	Devolución parcial por el flujo de Refund existente.
E8	Cambian de single a dobles después de reservar	Cambio de modalidad: se agregan o quitan lugares y se recalculan los lugares sin pagos. Solo si los lugares a quitar están vacíos.
E9	Modalidad sin precio cargado para no socio	No se puede reservar esa modalidad; la UI de precios lo marca como incompleto.
E10	Socio suspendido con reservas futuras	Sus partes ya calculadas se mantienen (RN-07).
E11	Socio dado de baja con cuenta de app	La cuenta sigue activa (es su identidad en Pique); simplemente paga como no socio.
E12	Club desactiva el módulo	Datos conservados; reservas nuevas a precio no socio.

10. Plan de implementación

Fase 0 — Base técnica (sin cambios visibles)

- Centralizar el cálculo de precio en `PricingService.resolveBookingPrice` con tests de regresión del modo por cancha.
- Migración de datos: normalizar DNI de usuarios, detectar duplicados.

Fase 1a — Identidad y padrón

- `User.email` opcional, `User.dni` único, `accountStatus`, `UserVerificationCode`.
- Login por DNI o email, activación por código (WhatsApp + código del mostrador), recuperación de contraseña.
- DNI opcional en registro y perfil, con bloqueo de DNI duplicado; ajuste de `claimClubProfile`.
- `onlineCheckoutEnabled = false` para el club.
- `MemberCategory`, `ClubMember`, `ClubMemberHistory`, ABM en admin, badge en Clientes, Agenda y Caja.

- Script de importación del padrón (crea `User` + `Client` + `ClubMember`).
- Gestión de la plantilla de WhatsApp de autenticación ante Meta (empezar temprano por los tiempos de aprobación).

Fase 1b — Precio por jugador y modalidades

- `pricingMode`, `ActivityFormat`, `PlayerPriceRule`, pantalla de precios.
- Lugares en `BookingParticipant`, validación de precio socio (D9), un ítem por jugador en la cuenta.
- Agenda: modalidad + lugares + validar socio en mostrador.
- App: modalidad, armado del partido, invitación por DNI, desglose de precios.
- Caja: cobro por jugador.

1a y 1b son ambas necesarias para la entrega. 1a puede probarse con el club (activación de cuentas, padrón) mientras se construye 1b.

Fase 1c — Cobro integrado con terminal

- **Paso previo (no técnico): relevar la terminal y el proveedor del club (P19).** Define si la integración es posible y con quién.
- `PaymentTerminal`, `TerminalPaymentIntent`, interfaz `InPersonPaymentProvider` y adaptador del proveedor del club.
- Webhook + polling de respaldo, registro automático del pago imputado al ítem del jugador.
- Caja: "Cobrar con terminal". Ajustes > Cobros: alta de terminales.
- Si la terminal del club no se puede integrar: cobro semiautomático (pago precargado + número de cupón) y evaluar con el club un cambio de terminal.

1c depende de 1b (necesita los ítems por jugador), pero el relevamiento de P19 tiene que arrancar **ya**, en paralelo con la Fase 0.

Fase 2

- Pantalla de importación de padrón.
- Descuentos por categoría en tienda, servicios y clases (`DiscountPolicy` vinculada a categoría).
- Informes socio / no socio y beneficio otorgado.
- Conciliación con el archivo de liquidación del proveedor.
- Pago online de la parte de cada jugador, si el club lo pide.

Fase 3 (si el club lo pide)

- Cuota social en Pique, con suspensión automática por morosidad (`validUntil` ya está preparado).
- Horarios o canchas exclusivos para socios, cupos, anticipación diferenciada.
- Grupo familiar.

---

11. Riesgos

Riesgo	Mitigación
Email opcional rompe flujos que asumen email	Revisión de los ~34 usos + tests de notificaciones, magic link, MercadoPago y OAuth.
Duplicados de DNI impiden el índice único	Migración previa con reporte y resolución manual.
Aprobación de la plantilla de WhatsApp demora	Pedirla al iniciar 1a; código del mostrador como alternativa.
Fraude con DNI de socios	D9 (confirmación) + RN-11 (un DNI, una cuenta) + RN-12 (no revelar socios).
Un ítem por jugador rompe supuestos de caja y reportes	Relevar todos los `type: 'BOOKING'` antes de empezar 1b; tests de caja.
Archivos muy grandes (`agenda-playground2.tsx` , `BookingService.ts`)	Cambios acotados; el motor de precios va a `PricingService` , la lógica de lugares a un servicio nuevo.
La terminal del club no tiene integración posible	Relevar P19 cuanto antes; plan B semiautomático; evaluar con el club cambiar de terminal.
Webhook del proveedor demorado o perdido	Polling de respaldo mientras la Caja espera + reconciliación de intenciones `SENT` viejas con un job.
Doble cobro (terminal + manual)	RN-T3: ítems bloqueados mientras hay una intención en curso.

12. Criterios de aceptación

Identidad - ☐ Un socio importado sin email activa su cuenta con DNI + código de WhatsApp y entra con DNI + contraseña. - ☐ Un socio sin teléfono activa su cuenta con un código generado en el mostrador. - ☐ Registrarse con el DNI de una cuenta existente no crea otra cuenta y no revela si es socio. - ☐ Recuperar contraseña por DNI funciona. - ☐ Los usuarios actuales siguen entrando con email.

Padrón - ☐ El script importa el padrón del club, reporta conflictos y no duplica clientes ni usuarios. - ☐ Alta, suspensión, reactivación y baja quedan en el historial con usuario, fecha y motivo.

Precio por jugador - ☐ Con `pricingMode = PER_COURT` , ningún precio cambia (regresión de Fase 0). - ☐ Tenis dobles con 1 socio (titular) y 3 no socios = precio socio dobles + 3 × no socio dobles. - ☐ Tenis single cotiza con los precios de single. - ☐ Un socio agregado por otro paga no socio hasta aceptar; al aceptar, su parte y el total bajan. - ☐ El staff puede validar a un socio en el mostrador y queda auditado. - ☐ La parte de un lugar con pagos no cambia al quitar al jugador, recalcular o cambiar el padrón. - ☐ En Caja se cobra a cada jugador solo su parte. - ☐ El mismo partido armado desde la app y desde la Agenda da el mismo precio. - ☐ Un turno con luces reparte el recargo en partes iguales entre los lugares.

Cobro con terminal - ☐ Desde Caja, cobrar la parte de un jugador envía el monto exacto a la terminal elegida. - ☐ Al aprobarse en la terminal, el pago aparece registrado e imputado al ítem del jugador sin intervención del cajero, en el turno de caja abierto. - ☐ Un webhook repetido no duplica el pago. - ☐ Un pago rechazado o cancelado no registra nada y permite reintentar. - ☐ Mientras hay un cobro en curso, ese ítem no se puede cobrar a mano. - ☐ Un monto aprobado distinto al esperado queda en revisión y no se registra solo.

No socios - ☐ Un no socio se registra sin DNI y reserva normalmente. - ☐ Un no socio que carga un DNI ya existente no queda vinculado a esa cuenta.

13. Preguntas al club

Respondidas

#	Pregunta	Respuesta
P3	¿Cuota social con Pique?	No por ahora; opción a futuro.
P4	Socios y no socios en el mismo partido	Cada jugador paga según su categoría.
P11	¿Precio por cancha o por jugador?	**Por jugador.** Ej.: no socio $6.000, socio $3.000. Es como trabaja este club.
P12	Modalidades	Tenis tiene single y dobles, con precios distintos administrados por el club.
—	Login	Por DNI. Los socios tendrán cuenta creada por nosotros a partir del padrón.
P2	¿El precio cambia por día u horario?	No. Solo el recargo por luces, que ya existe. `PlayerPriceRule` sin franjas (D10b).
P16	¿Pago online?	No. Solo en el mostrador (posnet o QR), sin seña; en el mejor caso cada jugador paga su parte. Quieren integrar las terminales para que el pago se registre solo (D12).
P17	¿No socios con DNI?	Se registran igual que hoy. El DNI se percibe invasivo → **opcional** para no socios (D4b).

Pendientes

#	Pregunta	Por qué importa
P19 ⚠️	¿Qué terminal (marca/modelo) y qué proveedor usan para el posnet? ¿El QR es de MercadoPago, de otra billetera o interoperable? ¿Fijo o generado por cobro? ¿Cuántas terminales y cajas?	Define si la integración automática es posible y con quién (D12, §6.7). **Es lo primero a relevar.**
P2b	El recargo por luces, ¿es por turno (se reparte entre los jugadores) o un monto por jugador?	Reparto en D10b.
P1	¿Qué categorías de socio tienen y qué precio tiene cada una?	Tabla de precios (D8).
P3b	¿Quién nos avisa altas, bajas y atrasos de socios? ¿Cada cuánto?	Si alcanza con ABM manual o conviene reimportar el padrón periódicamente.
P5	¿Hay beneficios que no sean de precio? (más anticipación, horarios exclusivos)	Fase 3.
P6	¿Descuentos para socios en buffet, tienda o clases?	Fase 2.
P8	¿El padrón tiene DNI y teléfono de todos los socios? ¿En qué formato está?	Importación y activación.
P10	¿Quién del club da de alta y baja socios?	Permisos (RN-09).
P13	¿Está bien que un jugador no vea la categoría de los demás, solo los montos?	Privacidad (§4.2).
P14	En turnos fijos, ¿juegan siempre los mismos?	Guardar jugadores en la serie.
P15	Si faltan jugadores por identificar, ¿quién paga esos lugares?	Comportamiento por defecto en Caja.
P18	¿Además de tenis y pádel hay otras actividades? ¿Cuántos jugadores tiene cada modalidad?	Carga inicial de modalidades.